Data and Algorithm Analysis

Chapter 14 — Dynamic Programming

Thang Hoang

Department of Computer Science, Virginia Tech

Outline

Overview

Rod Cutting

Paradigm

Matrix-Chain Multiplication

Longest Common String (LCS)

Table of Contents

Overview

Rod Cutting

Paradigm

Matrix-Chain Multiplication

Longest Common String (LCS)

Dynamic Programming

Not a specific algorithm, but a technique (like divide-and-conquer).

Developed back in the day when “programming” meant “tabular method”. Doesn’t really refer to computer programming.

Used for optimization problems:

- Find a solution that maximizes or minimizes some objective function.

Example — Traveling Salesperson Problem

TRAVELING SALESPERSON PROBLEM (TSP)

Input: Complete undirected graph $G = (V, E)$; weight function $w : E \to \mathbb{Z}$.

Output: A permutation $v_1, v_2, \ldots, v_n$ of V such that

$$w(v_n, v_1) + \sum_{i=1}^{n-1} w(v_i, v_{i+1})$$

is minimized.

- Solution is any permutation of V.
- Objective function to minimize is the given sum.

Other Examples

- Shortest path in a graph
- Minimum spanning tree
- Edit distance between strings
- Pattern matching
- Scheduling
- Maximum flow in a flow network
- Others

Dynamic Programming

Four-step method:

1. Characterize the structure of an optimal solution.
2. Recursively define the value of an optimal solution.
3. Compute the value of an optimal solution, typically in a *bottom-up* fashion.
4. Construct an optimal solution from computed information.

Table of Contents

Overview

Rod Cutting

Paradigm

Matrix-Chain Multiplication

Longest Common String (LCS)

Rod Cutting

- A company buys rods of length $n \in \mathbb{N}$.
- It cuts rods into integer-length pieces, which it sells.
- A rod of length i gets a price p_i.

ROD CUTTING

Input: Rod length n and prices $p_1, p_2, \ldots, p_n$.

Output: Positive integer rod lengths $i_1, i_2, \ldots, i_k$ such that

$$n = i_1 + i_2 + \cdots + i_k$$

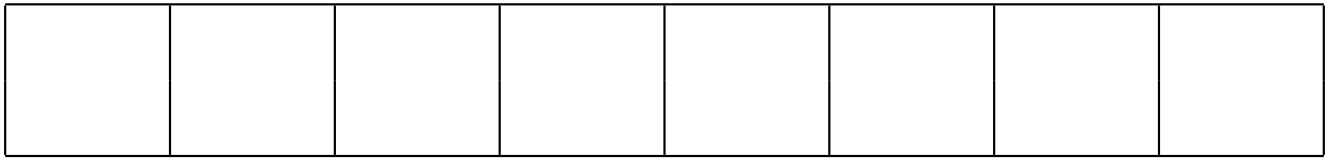
and

$$\sum_{j=1}^{k} p_{i_j}$$

is maximized.

Rod Cutting — Solution Space

Suppose the length of the initial rod is $n = 8$.



There are $n - 1 = 7$ places where a cut can occur.

So, how many different ways can this rod be cut into pieces?

For a general length n, what formula expresses the number of different ways to cut the length n rod into pieces?

Rod Cutting — Example Instance

Figure 14.1 contains this example of an instance for $n = 10$:

i	1	2	3	4	5	6	7	8	9	10
p_i	1	5	8	9	10	17	17	20	24	30

Optimal Substructure

To solve the original problem of size n, solve subproblems on smaller sizes.

After making a cut, two subproblems remain. The optimal solution to the original problem incorporates optimal solutions to the subproblems.

Optimal Substructure

Without loss of generality, assume that cuts are made left to right. For $0 \leq j \leq n$, let r_j be the optimal revenue for cutting a rod of length j. Suppose the first cut is at position i, where $1 \leq i \leq n$. Then, the optimal revenue for n is

$$p_i + r_{n-i}.$$

Since we do not know what i should be, we have

$$r_n = \max_{1 \leq i \leq n} p_i + r_{n-i}.$$

Recurrence

More generally, we get this recurrence for r_j, where $0 \leq j \leq n$:

$$r_j = \begin{cases} 0 & \text{if } j = 0; \\ \max\limits_{1 \leq i \leq j} p_i + r_{j-i} & \text{otherwise.} \end{cases}$$

By iterating from $j = 0$ to n, we can compute r_n.

The essence of dynamic programming is a recurrence that computes the optimal objective value in terms of optimal objective values for smaller instances.

Naive Recursive Implementation

```
NAIVE-CUT-ROD(p, n)
    if n == 0
        return 0
    q = -∞
    for i = 1 to n - 1
        q = max(q, p[i] + NAIVE-CUT-ROD(p, n - i))
    return q
```

Time complexity: considers all 2^{n-1} cuts explicitly, so $T(n) = \Omega(2^n)$.

Dynamic-Programming Solution

Instead of solving the same subproblems repeatedly, arrange to solve each subproblem just once.

Save the solution to a subproblem in a **table**, and refer back to the table whenever we revisit the subproblem.

- "Store, don't recompute!" $\Rightarrow$ time-memory trade-off.

Turn an exponential-time solution into a polynomial-time solution.

Two basic approaches:

1. Top-down with memoization.
2. Bottom-up.

Dynamic Programming — Top-Down with Memoization

Solve recursively, but store each result in a table.

To find the solution to a subproblem, first look in the table.

- If the answer is there, use it.
- Otherwise, compute the solution to the subproblem and then store the solution in the table for future use.

Dynamic Programming — Top-Down with Memoization

```
MEMOIZED-CUT-ROD(p, n)
let r[0:n] be a new array          // will remember solution values in r
for i = 0 to n
    r[i] = -∞
return MEMOIZED-CUT-ROD-AUX(p, n, r)
```

```
MEMOIZED-CUT-ROD-AUX(p, n, r)
if r[n] ≥ 0                 // already have a solution for length n?
    return r[n]
if n == 0
    q = 0
else q = -∞
    for i = 1 to n          // i is the position of the first cut
        q = max{q, p[i] + MEMOIZED-CUT-ROD-AUX(p, n - i, r)}
r[n] = q                    // remember the solution value for length n
return q
```

Dynamic Programming — Bottom-Up

Sort the subproblems by size and solve the smaller ones first.

```
BOTTOM-UP-CUT-ROD(p, n)
    Let r[0 : n] be a new array.
    r[0] = 0
    for j = 1 to n
        q = -∞
        for i = 1 to j
            q = max(q, p[i] + r[j - i])
        r[j] = q
    return r[n]
```

Time complexity: ?

Table of Contents

Overview

Rod Cutting

Paradigm

Matrix-Chain Multiplication

Longest Common String (LCS)

Dynamic Programming Paradigm

1. Identify subproblems and optimal substructure.
2. Develop a recurrence to compute the optimal objective values for each subproblem.
3. Compute a table of these values using the recurrence.
4. Backtrace to find the actual optimal solution.

Reconstructing a Solution

Save the optimal choices in a separate table and use a separate procedure to print them.

```
BOTTOM-UP-CUT-ROD(p, n)
    Let r[0 : n] and s[1 : n] be new arrays.
    r[0] = 0
    for j = 1 to n
        q = −∞
        for i = 1 to j
            if q < p[i] + r[j − i]
                q = p[i] + r[j − i]
                s[j] = i
        r[j] = q
    return r and s
```

Rod Cutting — Example Instance

i	0	1	2	3	4	5	6	7	8
p_i	0	1	5	8	9	10	17	17	20
r_i									
s_i									

Recurrence:

$$r_i = \begin{cases} 0 & \text{if } i = 0; \\ \max\limits_{1 \le j \le i} p_j + r_{i-j} & \text{otherwise.} \end{cases}$$

Rod Cutting — Computing the Table

i	0	1	2	3	4	5	6	7	8
p_i	0	1	5	8	9	10	17	17	20
r_i	0	1	5	8	10	13	17	18	22
s_i	0	1	2	3	2	2	6	1	2

Recurrence:

$$r_i = \begin{cases} 0 & \text{if } i = 0; \\ \max\limits_{1 \le j \le i} p_j + r_{i-j} & \text{otherwise.} \end{cases}$$

Table of Contents

Overview

Rod Cutting

Paradigm

Matrix-Chain Multiplication

Longest Common String (LCS)

Matrix-Chain Multiplication

Problem: Given a sequence (chain) of n matrices $(A_1, A_2, \ldots, A_n)$,
Compute the product $A_1 A_2 \ldots A_n$ using standard matrix multiplication (not Strassen's method),
while minimizing the number of scalar multiplications.

Cost of Multiplying Matrices

Suppose $A_1 \in \mathbb{F}^{10\times 100}$, $A_2 \in \mathbb{F}^{100\times 2}$, and $A_3 \in \mathbb{F}^{2\times 3}$.

- Multiplying $A_1 \times A_2$ requires $2 * 10 * 100 = 2000$ scalar multiplications.
- Multiplying $(A_1 \times A_2) \times A_3$ requires $2000 + 10 * 2 * 3 = 2060$ scalar multiplications.
- Changing the order of evaluation, multiplying $A_1 \times (A_2 \times A_3)$ requires $100 * 2 * 3 + 10 * 100 * 3 = 3600$ scalar multiplications.

Order of evaluation matters!

Optimization Problem

MATRIX CHAIN MULTIPLICATION

Input: Matrices $A_1, A_2, \ldots, A_n$ where A_i has dimensions $p_{i-1} \times p_i$.

Output: Parenthesization of $A_1 \times A_2 \times \cdots \times A_n$ that minimizes the number of scalar multiplications.

Solution could also be an **expression tree** — a binary tree with $\times$ at the internal nodes and matrices at the leaves.

Step 1: Structure of an Optimal Solution

Let $A_{i:j}$ be the matrix product $A_i A_{i+1} \ldots A_j$.

If $i < j$, then must split between A_k and A_{k+1} for some $i \leq k < j$ $\Rightarrow$ compute $A_{i:k}$ and $A_{k+1:j}$ and then multiply them together.

The total cost is

1. Cost of computing $A_{i:k}$ +
2. Cost of computing $A_{k+1:j}$ +
3. Cost of multiplying $A_{i:k}$ and $A_{k+1:j}$ together.

Step 1: Structure of an Optimal Solution

Optimal substructure: Suppose that optimal parenthesization of $A_{i:j}$ splits between A_k and A_{k+1}. Then the parenthesization of $A_{i:k}$ must be optimal.

Thus, to build an optimal solution to $A_{i:j}$, split it into how to optimally parenthesize $A_{i:k}$ and $A_{k+1:j}$, find optimal solutions to these subproblems, and then combine the optimal solutions.

Need to consider all possible splits.

Step 2: Construct a Recursive Solution

Let $m[i, j]$ be the minimum number of scalar multiplications to compute $A_{i:j}$.
Base cases: $m[i, i] = 0$, for $1 \leq i \leq n$.

General case: for $1 \leq i < j \leq n$,

$$m[i, j] \quad = \quad \min_{i \leq k < j} m[i, k] + m[k + 1, j] + p_{i-1} p_k p_j.$$

Define $s[i, j]$ to be a value of k to split $A_{i:j}$ in an optimal parenthesization. Then, $s[i, j] = k$ such that $m[i, j] = m[i, k] + m[k + 1, j] + p_{i-1} p_k p_j$.

Step 3: Compute the Optimal Costs

Use the tabular, bottom-up method

```
MATRIX-CHAIN-ORDER(p, n)
let m[1:n, 1:n] and s[1:n − 1, 2:n] be new tables
for i = 1 to n                          // chain length 1
    m[i, i] = 0
for l = 2 to n                          // l is the chain length
    for i = 1 to n − l + 1              // chain begins at A_i
        j = i + l − 1                   // chain ends at A_j
        m[i, j] = ∞
        for k = i to j − 1              // try A_{i:k} A_{k+1:j}
            q = m[i, k] + m[k + 1, j] + p_{i−1} p_k p_j
            if q < m[i, j]
                m[i, j] = q             // remember this cost
                s[i, j] = k             // remember this index
return m and s
```

Time complexity: ?

Step 4: Construct an Optimal Solution

With the s table filled in, recursively print an optimal solution.

```
PRINT-OPTIMAL-PARENS(s, i, j)
if i == j
    print "A"_i
else print "("
    PRINT-OPTIMAL-PARENS(s, i, s[i, j])
    PRINT-OPTIMAL-PARENS(s, s[i, j] + 1, j)
    print ")"
```

Example — Figure 14.5

There are $n = 6$ matrices with dimensions

p_0	p_1	p_2	p_3	p_4	p_5	p_6
30	35	15	5	10	20	25

Example — Table to Fill In

$m[i,j]$	1	2	3	4	5	6
1						
2						
3						
4						
5						
6						

Except for the base cases, for each $m[i,j]$ value, there is a k value that gives the root of an expression tree for that $m[i,j]$ value. This needs to be put in the table as well.

Example — Base Cases

$m[i,j]$	1	2	3	4	5	6
1	0					
2		0				
3			0			
4				0		
5					0	
6						0

Example — General Case — Next Diagonal

$m[i,j]$	1	2	3	4	5	6
1	0 –	15750 1				
2		0 –	2625 2			
3			0 –	750 3		
4				0 –	1000 4	
5					0 –	5000 5
6						0 –

Example — General Case — Finish

$m[i,j]$	1	2	3	4	5	6
1	0 —	15750 1	7875 1	9375 3	11875 3	15125 3
2		0 —	2625 2	4375 3	7125 3	10500 3
3			0 —	750 3	2500 3	5375 3
4				0 —	1000 4	3500 5
5					0 —	5000 5
6						0 —

Dynamic Programming Paradigm — Matrix Chain Multiplication

1. Identify subproblems and optimal substructure.
2. Develop a recurrence to compute the optimal objective values for each subproblem.
3. Compute a table of these values using the recurrence.
4. Backtrace to find the actual optimal solution.

Use backtrace in the previous example.
Time complexity to fill the table: $\Theta(n^3)$
Time complexity to find expression tree from the table: $\Theta(n)$

Table of Contents

Overview

Rod Cutting

Paradigm

Matrix-Chain Multiplication

Longest Common String (LCS)

Longest Common String (LCS)

Problem: Given two sequences, $X = (x_1, \ldots, x_m)$ and $Y = (y_1, \ldots, y_n)$.
Find a subsequence common to both whose length is longest.
A subsequence doesn't have to be consecutive, but it has to be **in order**.

Strings

A **string** is a sequence of characters from some finite alphabet Σ.

Example

DNA alphabet $\Sigma = \{\mathrm{A}, \mathrm{C}, \mathrm{G}, \mathrm{T}\}$. String

$$S = \mathrm{G}, \mathrm{G}, \mathrm{C}, \mathrm{A}, \mathrm{G}, \mathrm{T}, \mathrm{C}, \mathrm{T}$$

written

$$S = \mathrm{GGCAGTCT}$$

Length of S is 8. Empty string ϵ has length 0.

Substrings and Subsequences

A **substring** of a string $S = s_1 s_2 \cdots s_n$ is a string

$$S[i..j] \quad = \quad s_i s_{i+1} \cdots s_j.$$

A **subsequence** of S is a string

$$S' \quad = \quad s_{i_1} s_{i_2} \cdots s_{i_k},$$

where

$$1 \leq i_1 < i_2 < \cdots < i_k \leq n.$$

Example

Substrings of $S = \mathrm{GGCAGTCT}$ include S, ϵ, T, GCAG, but not GAT, which is a subsequence. Subsequences of S include all substrings plus CCT and GGGC.

Common Subsequence and LCS

Let $X = x_1 x_2 \cdots x_m$ and $Y = y_1 y_2 \cdots y_n$ be strings over Σ. The string $Z = z_1 z_2 \ldots z_k$ is a **common subsequence** of X and Y if it is a subsequence of both X and Y.
Z is a **longest common subsequence (LCS)** of X and Y if it is a common subsequence of maximum length.

Example

$$\begin{aligned} X &= \text{GGCAGTCT} \\ Y &= \text{TCTGATGC} \end{aligned}$$

TCT is a common subsequence of length 3.
What is an LCS of X and Y?

LCS Problem

LONGEST COMMON SUBSEQUENCE (LCS)

Input: Strings $X = x_1 x_2 \cdots x_m$ and $Y = y_1 y_2 \cdots y_n$ over Σ.

Output: String $Z = z_1 z_2 \ldots z_k$ that is a common subsequence of X and Y of maximum length.

Step 1: Characterize an LCS

Let $X_i = x_1 x_2 \cdots x_i$ be the prefix of X of length i.

Theorem

Let $X = x_1 x_2 \cdots x_m$ and $Y = y_1 y_2 \cdots y_n$. Let $Z = z_1 z_2 \ldots z_k$ be an LCS of X and Y.

1. *If $x_m = y_n$, then $z_k = x_m = y_n$ and Z_{k-1} is an LCS of X_{m-1} and Y_{n-1}.*
2. *If $x_m \neq y_n$ and $z_k \neq x_m$, then Z is an LCS of X_{m-1} and Y.*
3. *If $x_m \neq y_n$ and $z_k \neq y_n$, then Z is an LCS of X and Y_{n-1}.*

Proof.

In the textbook or on the board. □

Step 2: Recursively Define an Optimal Solution

Let $X = x_1 x_2 \cdots x_m$ and $Y = y_1 y_2 \cdots y_n$ be an instance of LCS.

Subinstances are pairs of all prefixes of X and Y, that is,

$$\begin{aligned} X_i &= x_1 x_2 \cdots x_i \\ Y_j &= y_1 y_2 \cdots y_j, \end{aligned}$$

where $0 \leq i \leq m$ and $0 \leq j \leq n$.

Define $c(i, j)$ = length of LCS of X_i and Y_j.

Recurrence for LCS

Base case(s): occur when $i = 0$ or $j = 0$:

$$\begin{aligned} c(i,0) &= 0 & 0 \leq i \leq m; \\ c(0,j) &= 0 & 0 \leq j \leq n. \end{aligned}$$

General cases: for $1 \leq i \leq m$ and $1 \leq j \leq n$:

$$c(i,j) = \begin{cases} c(i-1,j-1)+1 & \text{if } x_i = y_j; \\ \max\{c(i,j-1), c(i-1,j)\} & \text{if } x_i \neq y_j. \end{cases}$$

Recording the Choices for $c(i, j)$ with Arrows

$$c(i,j) = \begin{cases} c(i-1, j-1) + 1 & \text{if } x_i = y_j; \\ \max\{c(i, j-1), c(i-1, j)\} & \text{if } x_i \neq y_j. \end{cases}$$

The value of $c(i, j)$ depends directly on three other c values. The values that actually lead to a particular $c(i, j)$ value can be recorded with arrows in the table box.

	$j-1$	j
$i-1$	$c(i-1, j-1)$	$c(i-1, j)$
i	$c(i, j-1)$	$\nwarrow \quad \uparrow$ $\leftarrow c(i, j)$

Steps 3 & 4: Compute the Length of an LCS and Construct an LCS

See pseudocode of LCS-LENGTH(X, Y, m, n) and PRINT-LCS(b, X, m, n) in book.

Example — Empty Table

$c(i,j)$		0	G 1	A 2	C 3	G 4	C 5	A 6
	0							
C	1							
A	2							
G	3							
A	4							
G	5							

Example — Base Cases

$c(i,j)$		0	G 1	A 2	C 3	G 4	C 5	A 6
	0	0	$\leftarrow$ 0	$\leftarrow$ 0	$\leftarrow$ 0	$\leftarrow$ 0	$\leftarrow$ 0	$\leftarrow$ 0
C	1	$\uparrow$ 0						
A	2	$\uparrow$ 0						
G	3	$\uparrow$ 0						
A	4	$\uparrow$ 0						
G	5	$\uparrow$ 0						

Example — General Case — Row $i = 1$

$c(i,j)$		0	G 1	A 2	C 3	G 4	C 5	A 6
	0	0	$\leftarrow$ 0	$\leftarrow$ 0	$\leftarrow$ 0	$\leftarrow$ 0	$\leftarrow$ 0	$\leftarrow$ 0
C	1	$\uparrow$ 0	$\uparrow$ $\leftarrow$0	$\uparrow$ $\leftarrow$0	$\nwarrow$ 1	$\leftarrow$ 1	$\nwarrow$ $\leftarrow$ 1	$\leftarrow$ 1
A	2	$\uparrow$ 0						
G	3	$\uparrow$ 0						
A	4	$\uparrow$ 0						
G	5	$\uparrow$ 0						

Example — General Case — Complete Table

$c(i,j)$		0	G 1	A 2	C 3	G 4	C 5	A 6
	0	0	← 0	← 0	← 0	← 0	← 0	← 0
C	1	↑ 0	↑ ←0	↑ ←0	↖ 1	← 1	↖ ← 1	← 1
A	2	↑ 0	↑ ←0	↖ 1	↑ ←1	↑ ←1	↑ ←1	↖ 2
G	3	↑ 0	↖ 1	↑ ←1	↑ ←1	↖ 2	← 2	↑ ←2
A	4	↑ 0	↑ 1	↖ 2	← 2	↑ ←2	↑ ←2	↖ 3
G	5	↑ 0	↖↑ 1	↑ 2	↑ ←2	↖ 3	← 3	↑ ←3

Example — Backtrace — Get LCS AGA

$c(i,j)$		0	G 1	A 2	C 3	G 4	C 5	A 6
	0	0	← 0	← 0	← 0	← 0	← 0	← 0
C	1	↑ 0	↑ ←0	↑ ←0	↖ 1	← 1	↖ ← 1	← 1
A	2	↑ 0	↑ ←0	↖ 1	↑ ←1	↑ ←1	↑ ←1	↖ 2
G	3	↑ 0	↖ 1	↑ ←1	↑ ←1	↖ 2	← 2	↑ ←2
A	4	↑ 0	↑ 1	↖ 2	← 2	↑ ←2	↑ ←2	↖ 3
G	5	↑ 0	↖↑ 1	↑ 2	↑ ←2	↖ 3	← 3	↑ ←3

Dynamic Programming Paradigm — LCS

1. Identify subproblems and optimal substructure.
2. Develop a recurrence to compute the optimal objective values for each subproblem.
3. Compute a table of these values using the recurrence.
4. Backtrace to find the actual optimal solution.

Time complexity to fill the table: $\Theta(mn)$
Time complexity to find one LCS from the table: $\Theta(m+n)$